Arduino Control Example

This section aims to guide you on how to use the PC serial port to control MiniArm's servo angles, buzzer, and RGB lights, as well as how to use it to read servo angles.

1. Working Principle

The source code of the program is located at: "\1.Tutorials\9.Serial Communication Instruction\arduino_and_MiniArm\arduino_and_MiniArm.ino". For specific implementation details, please access the file: "\1.Tutorials\9.Serial Communication Instruction\arduino_and_MiniArm MiniArm_ctl.cpp".

1) Set the baud rate of debugging and printing serial port as 115200, and call the "mi_ctl.serial_begin()" function in the "MiniArm_ctl.h" library to initialize the software serial port for communication with a baud rate of 9600.

```
void setup() {
  Serial.begin(115200);
  uh_ctl.serial_begin();
}
```

2) Call the "mi_ctl.set_angles()" function to control the servo angles. Take the "mi_ctl.set_angles(angles)" code as an example. The first parameter "angles" represents each servo's rotating angle. This parameter is a one-dimensional array with a length of 5 corresponding to 5 servos' required rotation angles respectively, with a rotation range from 0°to 180°.

```
//set servos' angles
Serial.println("set angles");
uint8_t angles[5] = {73,10,161,57,90};
/* set respectively the bus servos' angles to 73/10/0/57/90/180 digresses,
  ranging from 0-180 */
mi_ctl.set_angles(angles);
delay(1500);
```

3) Call the "mi_ctl.set_buzzer()" function to set the buzzer. Take the

“mi_ctl.set_buzzer(532, 1000)” code as an example. The first parameter “532” indicates the buzzer frequency to be set ranging from 0 to 65535. The second parameter “1000” represents the running time (in ms) which is 1000ms.

```
//set buzzer
Serial.println("set buzzer");
//set the buzzer frequency to 532 and the runtime to 1000 ms
//the ranges of these two parameters are both 0-65535
mi_ctl.set_buzzer(532,1000);
delay(1500);
```

4) Call the “mi_ctl.set_rgb()” function to control the color of RGB light. Take the “mi_ctl.set_rgb(rgb)” code as an example. The parameter “rgb” is a one-dimensional array with a length of three, corresponding to red, green, and blue respectively, and ranging from 0 to 255.

```
//set RGB light
Serial.println("set rgb");
uint8_t rgb[3] = {0,255,0};
//set RGB light to green (ranging from 0 to 255)
mi_ctl.set_rgb(rgb);
delay(1500);
```

5) Call the “mi_ctl.read_angles()” function to obtain the angle information of 5 servos of MiniArm. The read servo angles can be saved in the “read_a” array and printed via the serial port.

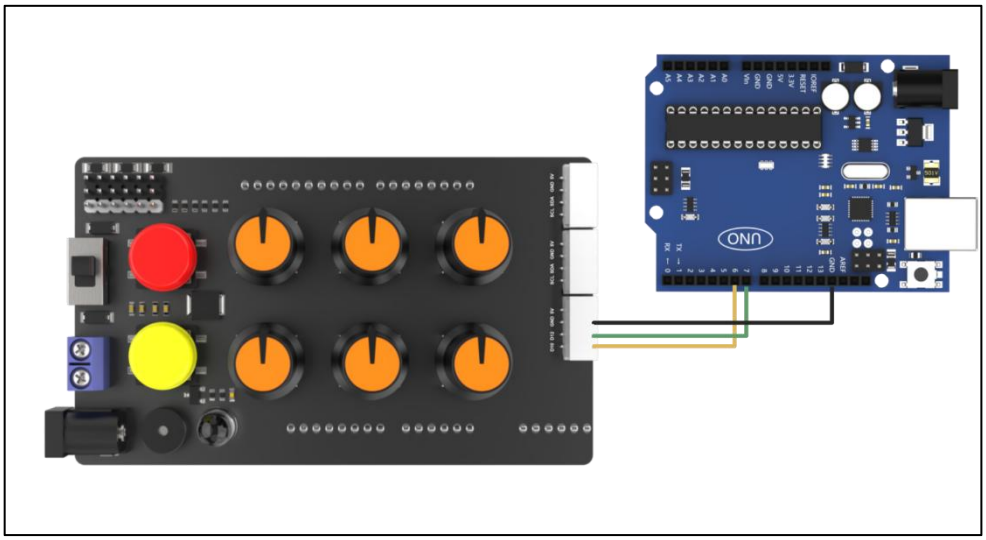
```
//read and print the servo angle of current status
Serial.println("read angles");
uint8_t read_a[5] = {0,0,0,0,0};
if(mi_ctl.read_angles(read_a))
{
  Serial.print("angles: ");
  for(int i=0;i<5;i++){
    Serial.print(read_a[i]);
    Serial.print(" ");
  }
}
delay(100);
```

2. Getting Ready

2.1 Hardware

The interface for MiniArm serial communication is a 4-pin. It is suitable for devices with pin interfaces for UART communication such as Arduino. Now you need to connect the MiniArm's 4-pin to its corresponding pin interfaces for communication.

The connection of the 4-pin interface communication is as follows:



The detailed connection is as follows:

Connect the IO12 pin of MiniArm used as the TX signal pin to the No.6 pin of Arduino.

Connect the IO10 pin of MiniArm used as the RX signal pin to the No.7 pin of Arduino.

Connect the GND pins of MiniArm and Arduino.

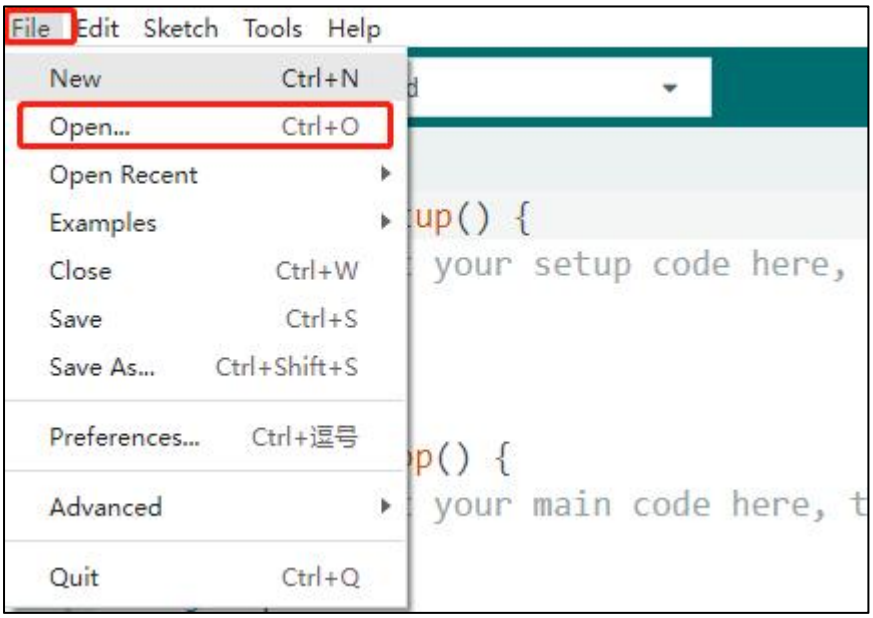
2.2 Software

Please refer to the instructions in "2.Set Arduino Environment" to establish the connection between MiniArm and Arduino editor.

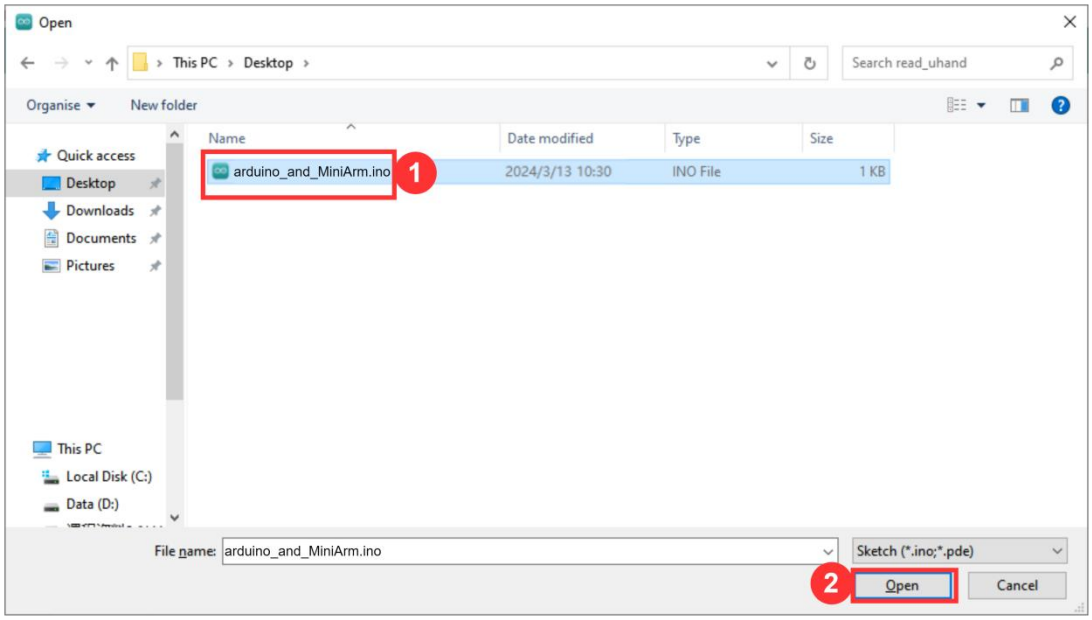
3. Program Download

1) Double-click to open it.

2) Click “File -> Open”.

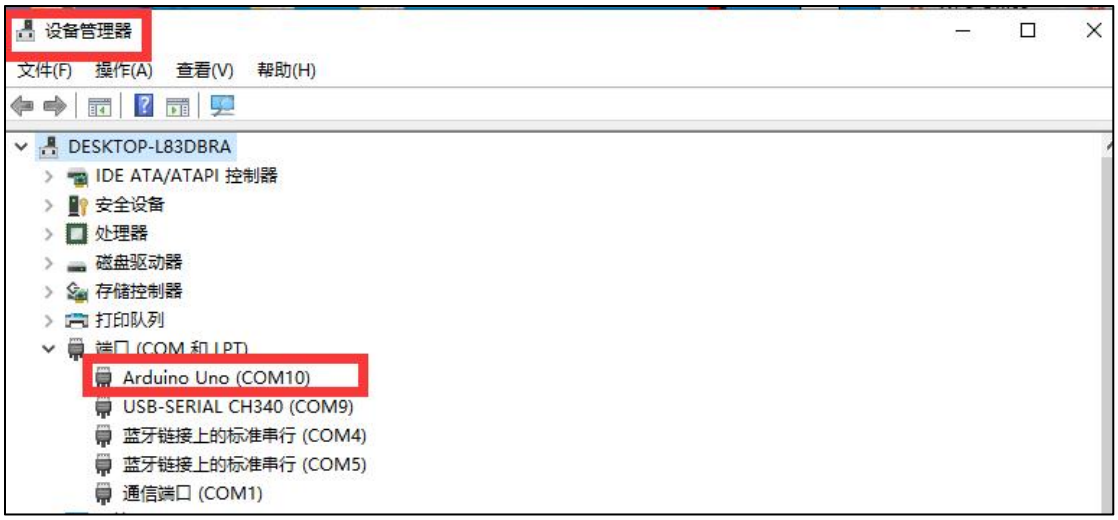


3) Open the source code path in “1. Working Principle” to click “Open”.

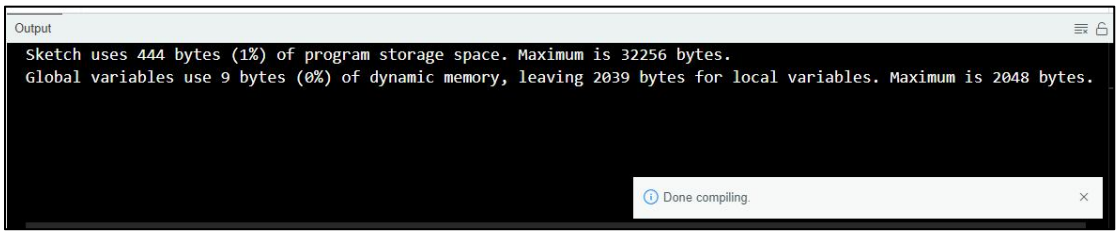


4) Click “Tools -> Development Board” to select “Arduino UNO”. Take COM10 as an example. (If you have already configured the development board model and port number during the setup of the development environment, please skip this step.)

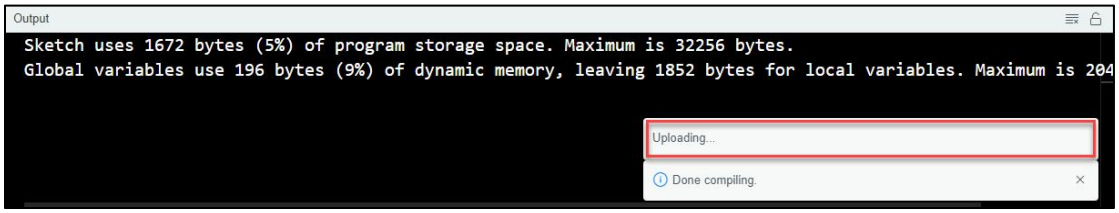
5) If you are unsure about the port number, please right-click “This PC” and click “Properties -> Device Manager” to view the port number corresponding to the master (the device with CH340). Once you have found it, you can reselect it in the Arduino.



6) After confirming that everything is correct, click to verify the program. If the program is correct, the status area shows “Compiling Project -> Completed” in turn. Then it displays the number of bytes used by the current project, the program storage space, and etc.



7) After successful compilation, click to upload the program to the development board. The status area shows “Compiling Project -> Uploading -> Upload successfully” in turn. After that, the status area stops printing and uploading information.



4. Program Outcome

When the program runs:

1) Five servos respectively rotate to 73, 10, 161, 57, and 90 degrees.

2) After 1.5 seconds, the buzzer of MiniArm sounds at 532hz for 1 second.

3) Then, the color of the LED is set to green.

4) Finally, the angles of MiniArm's 5 servos are printed on the serial monitor.